



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

第五章：死锁

夏文 教授

哈尔滨工业大学（深圳）

2026年秋季

Email: xiawen@hit.edu.cn

- 死锁 (Deadlock) 的引入
- 死锁特征分析—产生死锁的四个必要条件
- 死锁处理方法

生活中的死锁

- **案例一：**上周，我手机被我摔坏了，因为急用，就想去益田假日买一部新的。这时候保安拦住了我，由于深圳近期疫情，他让我出示健康码。我说我手机摔坏了，要进去买一部新手机才能出示健康码；他说，最近疫情严重了，要出示健康码才能进去买手机。。。哇，多么完美的死锁！
- **案例二：**最近小刘在办离职手续，HR让小刘去人才市场开调档函，把他的档案调过去，然后才能办离职证明。当小刘去到人才市场，人才市场的工作人员告诉小刘，要拿着离职证明过来才能开调档函。。。又双叒出现了，多么熟悉的死锁！



信号量产生死锁

```
1. volatile long cnt = 0; /* Counter */
2. sem_t mutex[2]; /* Semaphore that pr
3. int NITERS;

5. int main()
6. {
7.     pthread_t tid[2];

9.     NITERS = atoi(argv[1]);

11.     Sem_init(&mutex[0], 0, 1); /* mutex[0] = 1 */
12.     Sem_init(&mutex[1], 0, 1); /* mutex[1] = 1 */
13.     Pthread_create(&tid[0], NULL, count, (void*) 0);
14.     Pthread_create(&tid[1], NULL, count, (void*) 1);
15.     Pthread_join(tid[0], NULL);
16.     Pthread_join(tid[1], NULL);
17.     printf("cnt=%d\n", cnt);
18.     exit(0);
19. }
```

deadlock.c

```
20. void *count(void *vargp)
21. {
22.     int i;
23.     int id = (int) vargp;
24.     for (i = 0; i < NITERS; i++) {
25.         P(&mutex[id]); P(&mutex[1-id]);
26.         cnt++;
27.         V(&mutex[id]); V(&mutex[1-id]);
28.     }
29.     return NULL;
30. }
```

Tid[0]:

P(s₀);
P(s₁);
cnt++;
V(s₀);
V(s₁);

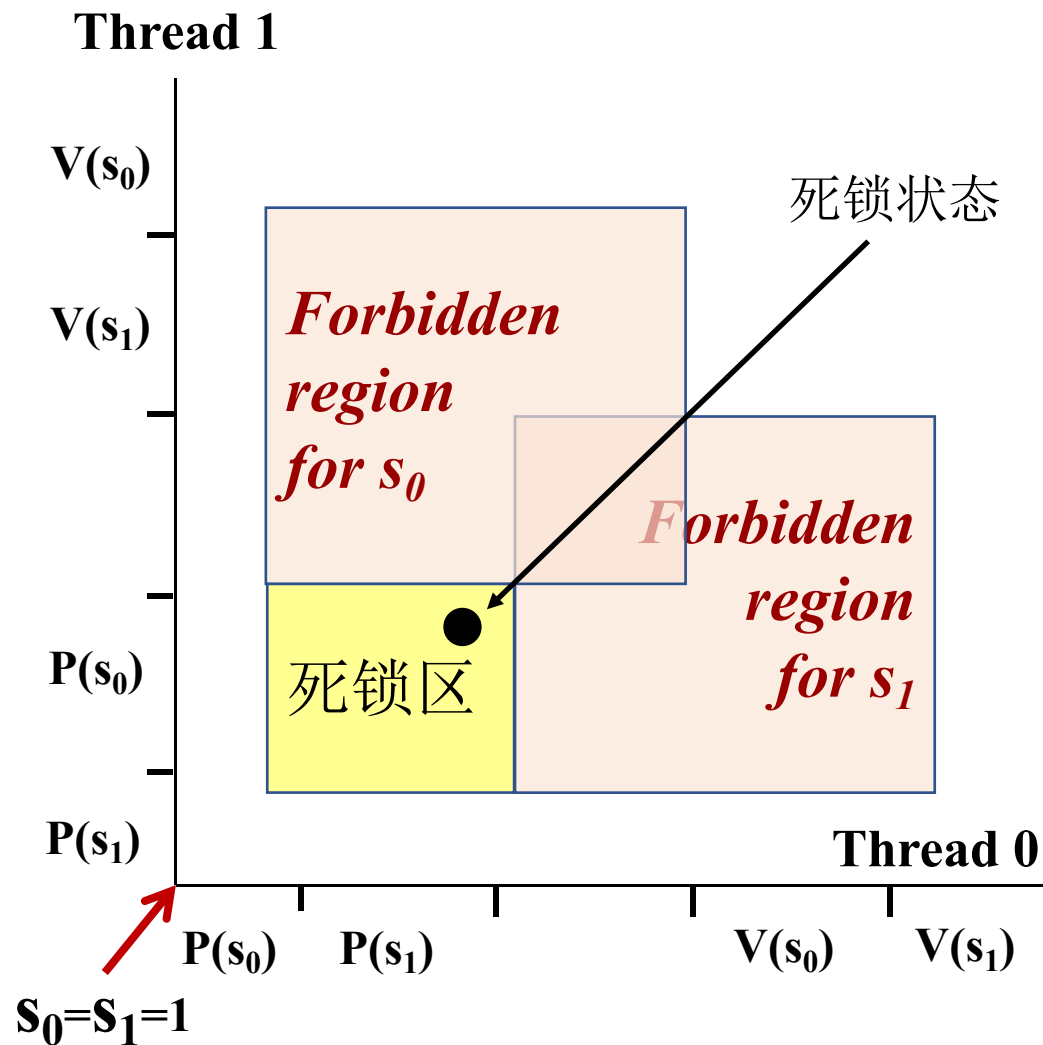
Tid[1]:

P(s₁);
P(s₀);
cnt++;
V(s₁);
V(s₀);

lpsh@ubuntu:~/Desktop/Files\$./deadlock 1000
cnt=2000 正确执行

lpsh@ubuntu:~/Desktop/Files\$./deadlock 1000
发生死锁

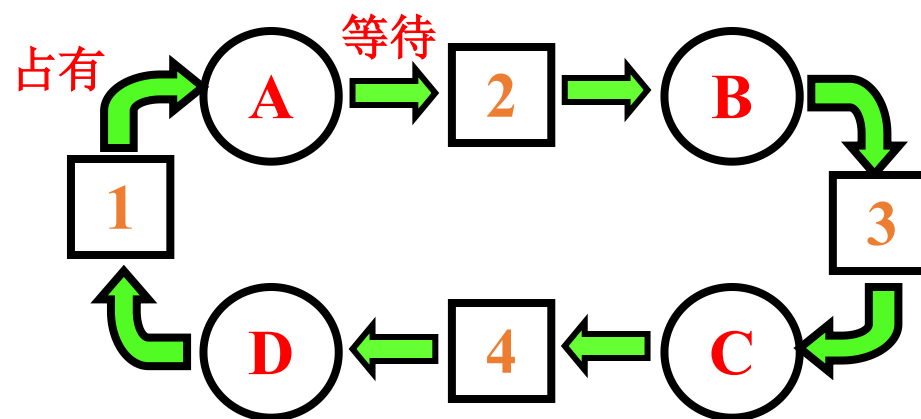
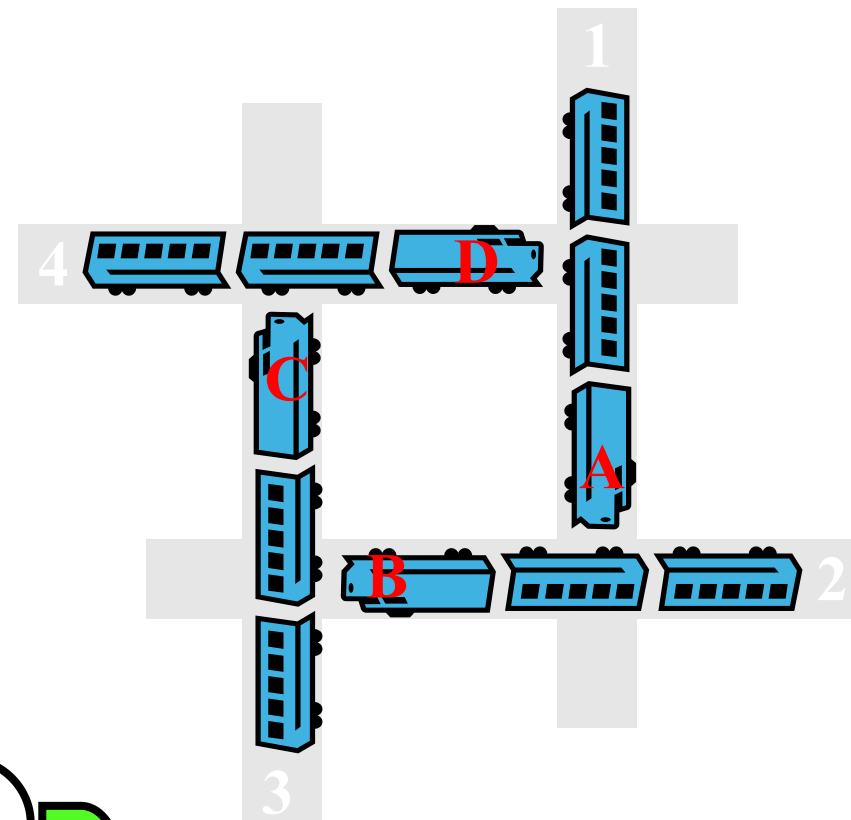
利用进度图可视分析死锁



- 上锁带来了潜在的**死锁 (deadlock)**：即等待一个永远不会为真的条件。
- 任何进入**死锁区 (deadlock region)** 的进程推进顺序都会最终进入**死锁状态 (deadlock state)**，等待 s_0 或 s_1 变为非0。
- 其他进程推进顺序则可以避免进入死锁区。
- 比较棘手的是，死锁经常是非确定的 (nondeterministic)
- 不同的执行轨迹会造成不同的执行结果——死锁的产生与调度有关！

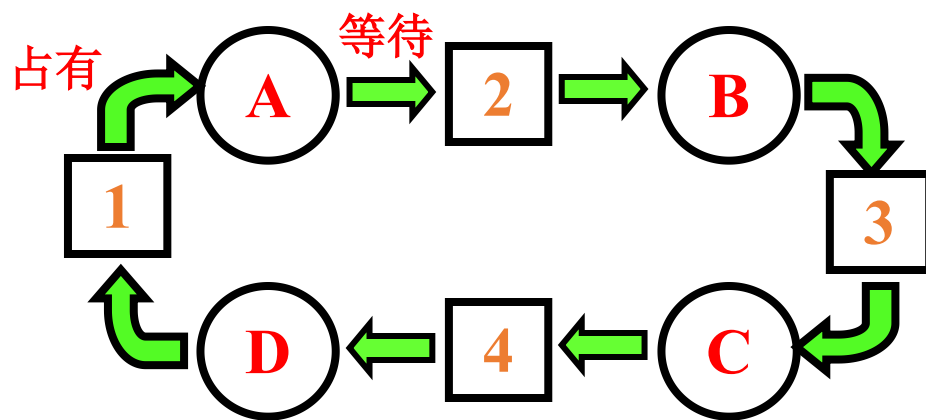
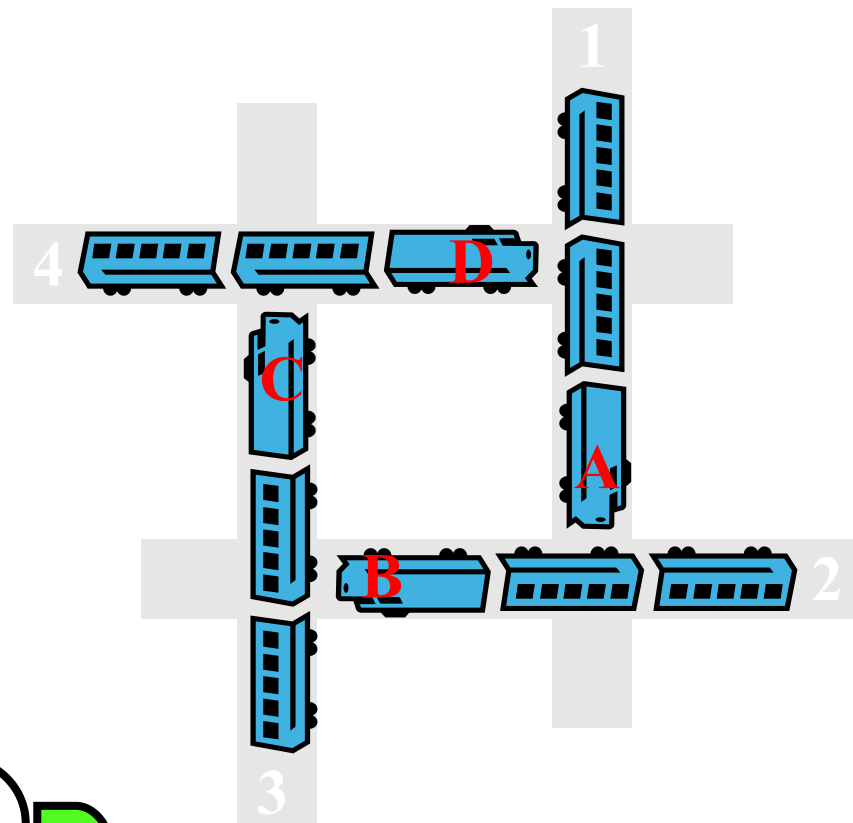
死锁现象

- 看一个实际的例子
- 竞争使用资源: 道路
- A占有道路1, 又要请求道路2, B占有...
- 形成了无限等待



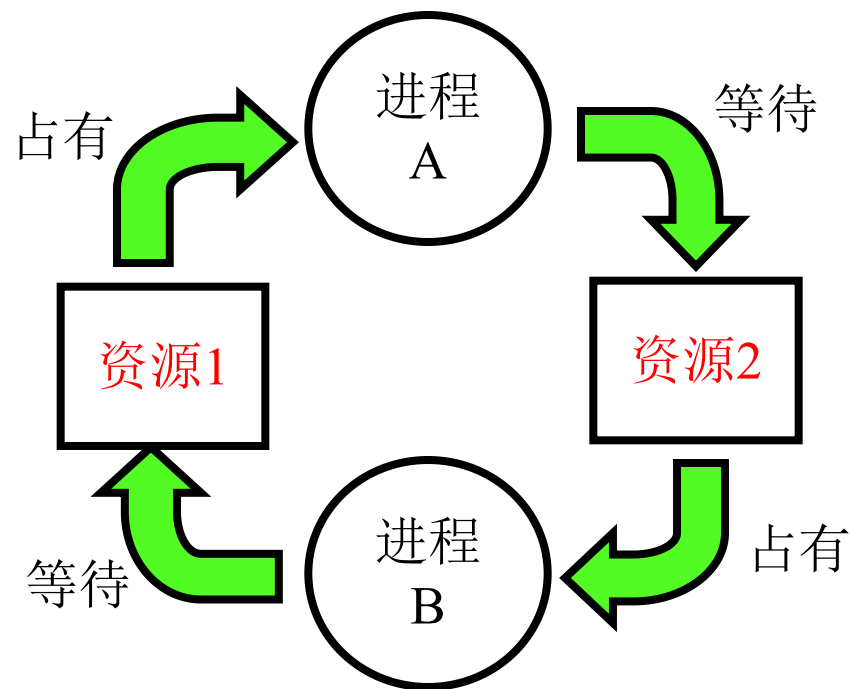
死锁特征

- 每条道路（资源）只能由一辆火车（进程）占有。
- 某条道路由一火车占用后其他火车不能抢占前者的道路。
- 火车持续持有所占有的道路，等待其他道路空闲。
- 火车之间相互等待。



死锁概念(Deadlock)

- 死锁: 多个进程（线程）因循环等待资源而造成无法执行的现象。
- 死锁会造成进程（线程）无法执行
- 死锁会造成系统资源的极大浪费(资源无法释放)



- 死锁 (Deadlock) 的引入
- 死锁特征分析—产生死锁的四个必要条件
- 死锁处理方法

资源的分析

■ 多个进程因等待资源才造成死锁

■ 资源: 进程在完成其任务过程所需要的所有对象

➤ CPU、内存、磁盘块、外设、文件、信号量 ...

■ 显然有些资源不会造成死锁，而有些会

➤ 只读文件是不会造成进程等待的，也就不会死锁

➤ 打印机一次只能让一个进程使用，就会造成死锁

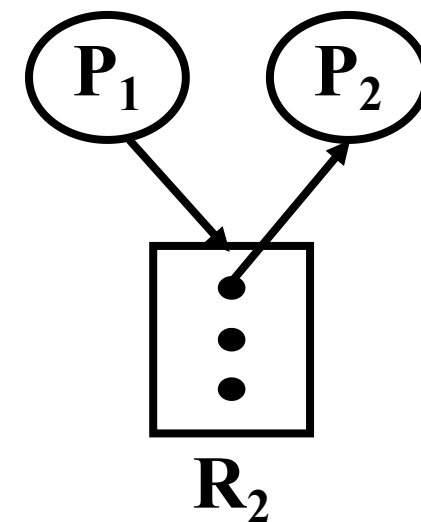
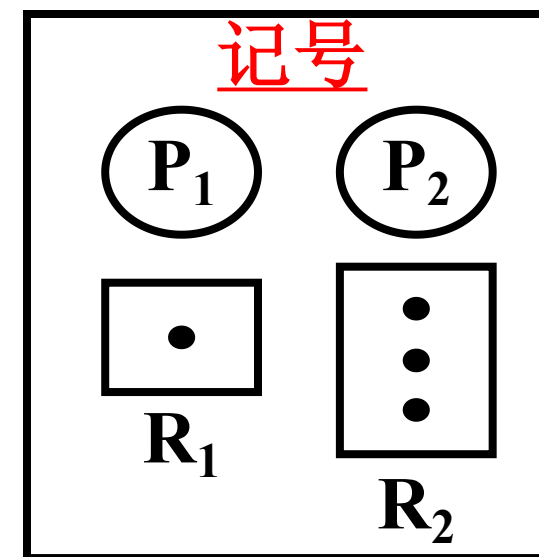
称为互斥访问资源

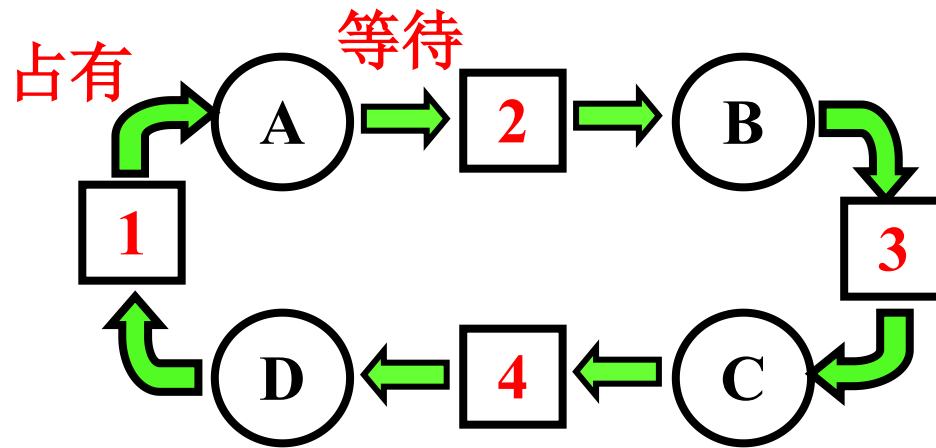
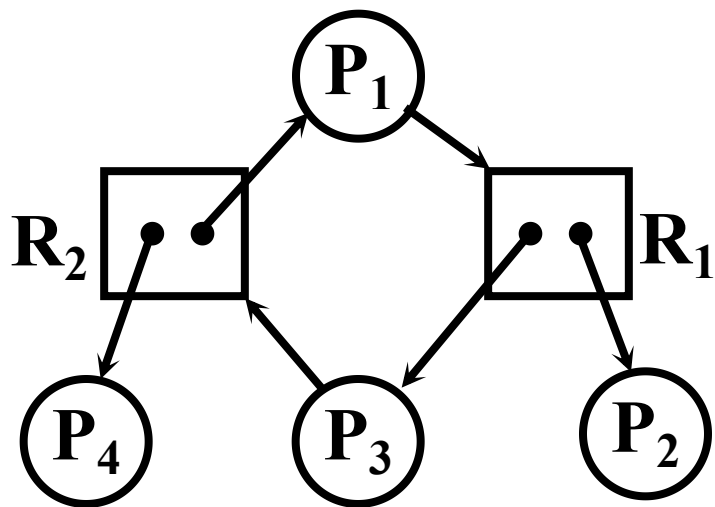
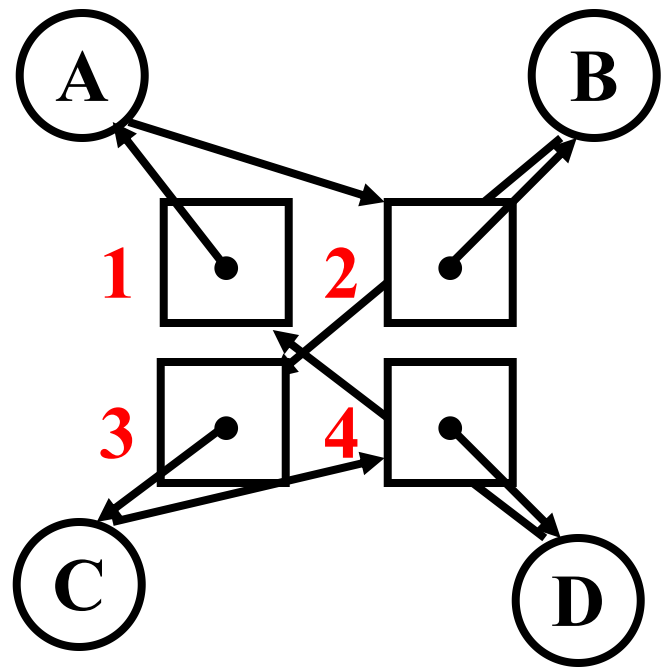
➤ 显然，资源互斥访问是死锁的必要条件

■ 资源请求需要形成环路等待才死锁！如何描述这种等待关系？

资源分配图模型

- 一个进程集合 $\{P_1, P_2, \dots, P_n\}$
- 一资源类型集合 $\{R_1, R_2, \dots, R_m\}$
- 资源类型 R_i 有 W_i 个实例
- 资源请求边：有向边 $P_i \rightarrow R_j$
- 资源分配边：有向边 $R_i \rightarrow P_k$





■ 存在环路: $1 \rightarrow A \rightarrow 2 \rightarrow B \rightarrow 3 \rightarrow C \rightarrow 4 \rightarrow D \rightarrow 1$
(产生死锁)

■ 存在环路: $P1 \rightarrow R1 \rightarrow P3 \rightarrow R2 \rightarrow P1$
(但并不死锁, 仍可继续执行, 其中R2: $W_2 > 1$)

死锁的4个必要条件

■ 1. 互斥使用(Mutual exclusion)

- 至少有一个资源互斥使用

■ 2. 不可抢占(No preemption)

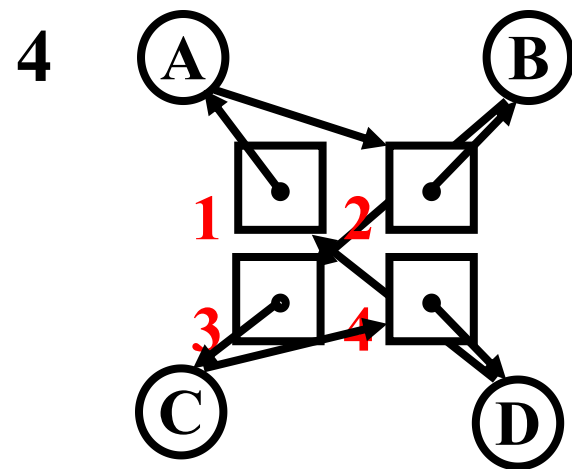
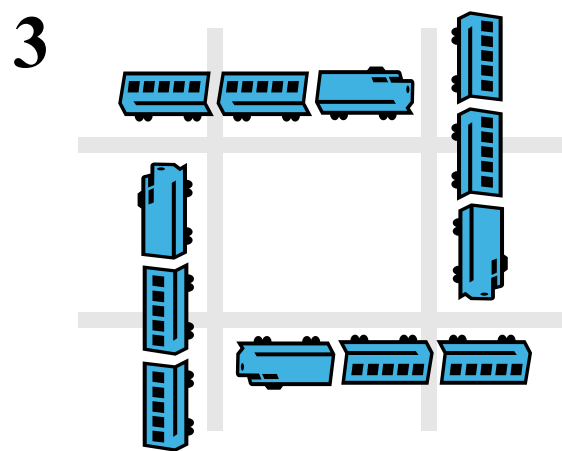
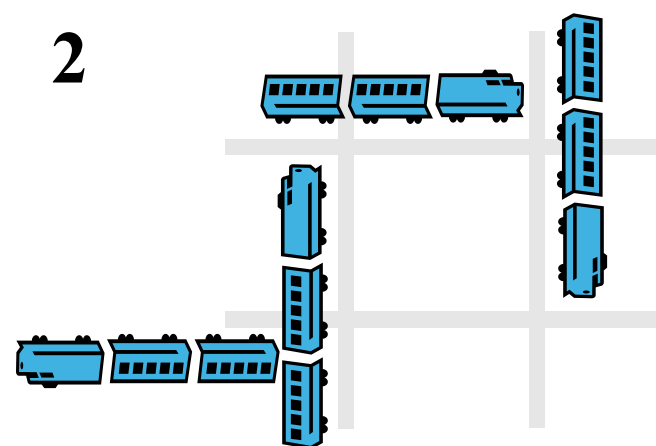
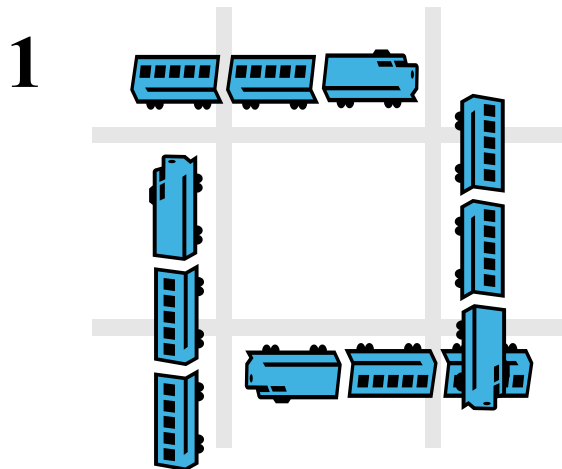
- 资源只能自愿放弃，如车开走以后

■ 3. 请求和保持(Hold and wait)

- 进程必须占有资源，再去申请

■ 4. 循环等待(Circular wait)

- 在资源分配图中存在一个环路



- 死锁 (Deadlock) 的引入
- 死锁特征分析—产生死锁的四个必要条件
- 死锁处理方法



死锁处理方法概述

■ 1. 死锁预防：有河水的地方不走

- 破坏死锁的必要条件

■ 2. 死锁避免：遇到河水了，就摸石头过河，没有石头的地方不走

- 检测每个资源请求，如果造成死锁就拒绝

■ 3. 死锁检测+恢复：掉坑里了，再爬出来

- 检测到死锁出现时，剥夺一些进程的资源

■ 4. 死锁忽略：躺平

- 就好像没有出现死锁一样

死锁处理方法概述

死锁处理策略：比喻与技术对比

1. 死锁预防



有河水的地方不走

🔗 破坏死锁的必要条件

2. 死锁避免



遇到河水了，就摸石头过河，没有石头的地方不走

🔍 检测每个资源请求，如果造成死锁就拒绝

3. 死锁检测+恢复



📶 掉坑里了，再爬出来

🔧 检测到死锁出现时，剥夺一些进程的资源

4. 死锁忽略



😴 躺平

🙄 就好像没有出现死锁一样

死锁预防: 破除死锁的必要条件之(1)(2)

■ 破坏互斥使用

- 资源的固有特性，通常无法破除，如打印机

■ 破除不可抢占

- 如果一个进程占有资源并申请另一个不能立即分配的资源，那么已分配资源就可被抢占（即持有不用即可被抢占）
- 如果申请的资源得到满足，则抢占其他资源一次性分配给该进程
- 只对状态能保存和恢复的资源(如CPU，内存空间)有效，对打印机等外设不适用

■ 实例：两个进程使用串口，都要读串口（并操作），数据不同不可恢复。

■ 破除请求和保持

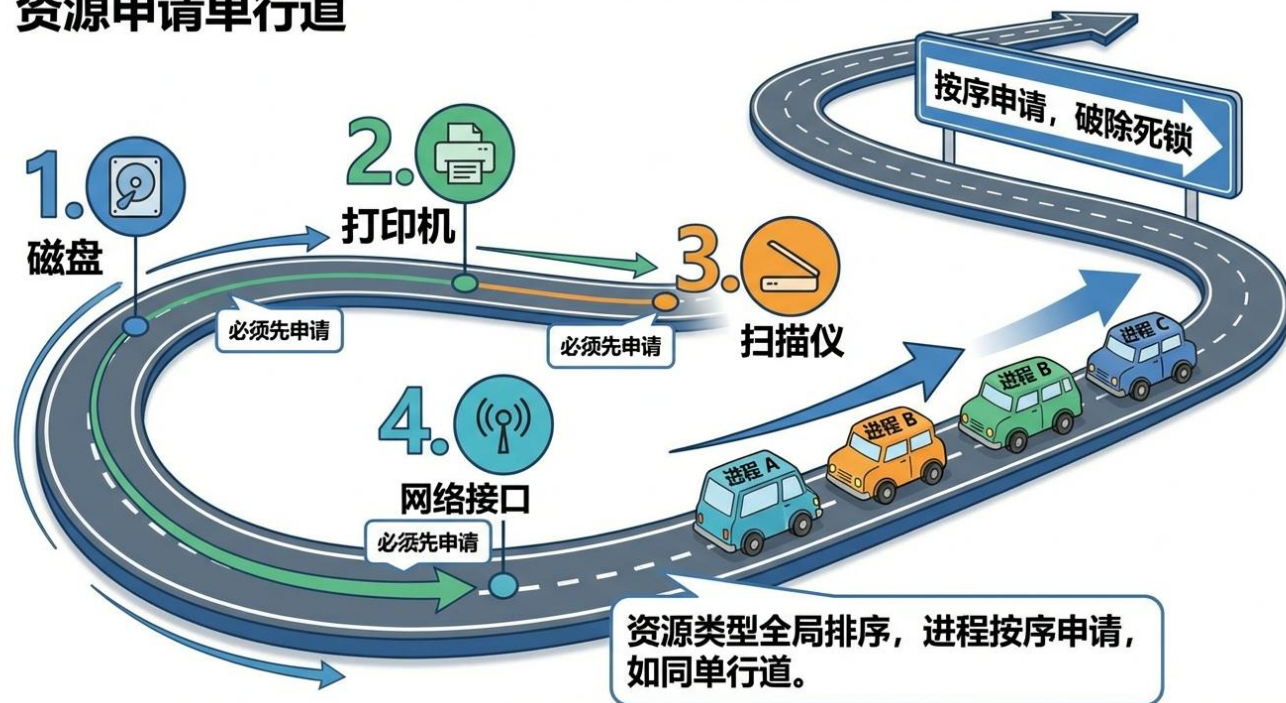
- 在进程执行前，一次性申请所有需要的资源
- 缺点1: 需要预知未来，编程困难
- 缺点2: 许多资源分配后很长时间后才使用，资源利用率低

死锁预防: 破除死锁的必要条件之(4)

■ 破除循环等待 (必要条件4)

- 对资源类型进行排序, **资源申请必须按序进行**。例如: 所有的进程必须先申请磁盘驱动, 再申请打印机, 再..., 如同交通中的单行道。
- 缺点: 如果**编程时就需考虑**, 用户会觉得很别扭; 为了去申请序号小的资源, 要先释放某些序号高的资源, 导致**执行效率降低**。
- 在通用操作系统中使用不多, 但在**嵌入式操作系统**中有使用。

资源申请单行道



- 总之, **破除死锁的必要条件会引入不合理因素, 实际中很少使用。**

- 思想: 判断此次请求**是否造成死锁**若会造成死锁, 则拒绝该请求

不死锁就成了问题的核心!

- **安全状态定义**: 如果系统中的所有进程存在一个可完成的执行序列 $P_1, \dots, P_n$, 则称系统处于安全状态

都能执行完成当然就不死锁

- **安全序列**: 上面的执行序列 $P_1, \dots, P_n$

如何找到这样的序列?

死锁避免之银行家算法

■ 一个银行家：目前手里只有1亿

- 第A个开发商：已贷款15亿，资金紧张还需3亿。
- 第B个开发商：已贷款5亿，还需贷款1亿，运转良好能收回。
- 第C个开发商：已贷款2亿，欲贷款18亿。

■ 开发商B还钱，再借给A，则可以继续借给C

■ 银行家当前可用的资金（**Available**）？可以利用的资金，即可用的加上能收回的共有多少（**Work**）？各个开发商已贷款——已分配的资金（**Allocation**）？各个开发商还需要贷款（**Need**）

■ 安全序列 $P_1, \dots, P_n$ 应该满足的性质:

$P_i (1 \leq i \leq n)$ 需要资源 $\leq$ 剩余资源 + 分配给 $P_j (1 \leq j \leq i)$ 资源

```
1. Banker();  
2. int n, m; // 系统中进程总数n和资源种类总数m  
3. int Available[m]; // 资源当前可用总量  
4. int Allocation[n][m]; // 当前给分配给每个进程的各种资源数量  
5. int Need[n][m]; // 当前每个进程还需分配的各种资源数量  
6. int Work[m]; // 当前可分配的资源, 包括可收回的  
7. bool Finish[n]; // 进程是否结束
```

■ 安全状态判定（思路）：

➤ ①初始化设定：

$Work = Available$ （动态记录当前可（收回）分配资源）

$Finish[i] = false$ （设定所有进程均未完成）

➤ ②查找这样的进程 P_i （未完成但目前剩余资源可满足其需要，这样的进程是能够完成的）：

a) $Finish[i] == false$ && b) $Need[i] \leq Work$

如果没有这样的进程 P_i ，则跳转到第④步

➤ ③（若有则） P_i 一定能完成，并归还其占用的资源，即：

a) $Finish[i] = true$ b) $Work = Work + Allocation[i]$

GOTO 第②步，继续查找

➤ ④如果所有进程 P_i 都是能完成的，即 $Finish[i] = true$

则系统处于安全状态，否则系统处于不安全状态

■ 当前状态:

	Work=[3 3 2]
P_1	Work=[5 3 2]
P_3	Work=[7 4 3]
P_4	Work=[7 4 5]
P_0	Work=[7 5 5]
P_2	Work=[10 5 7]

	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	3 3 2
P_1	2 0 0	1 2 2	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

安全序列是 $\langle P_1, P_3, P_4, P_0, P_2 \rangle$

安全序列是唯一的吗?

```
1. bool Found;  
2. Work = Available; Finish = false;  
3. while(true) {  
4.     Found = false; //是否为安全序列找到一个新进程  
5.     for(i=1; i<=n; i++) {  
6.         if(Finish[i]==false && Need[i]<=Work) {  
7.             Work = Work + Allocation[i];  
8.             Finish[i] = true;  
9.             printf("%d->", i); //输出安全序列  
10.            Found = true;  
11.        }  
12.    } 没有安全序列或已经找到  
13.    if(Found==false) break;  
14. }  
15. for(i=1; i<=n; i++)  
16.     if(Finish[i]==false)  
17.         return "deadlock";
```

$$T(n)=O(mn^2)$$

最好情形：安全状态就是 P_1-P_n

最坏情形： P_n-P_1

死锁避免之资源请求算法

■ 思想：可用的资源可以满足某个进程的**当前资源请求Request**，则分配，然后寻找安全序列，找到，分配成功，找不到，已分配资源收回。

```
1. extern Banker();
2. int Request[m]; /*进程Pi的资源申请*/
3. if(Request>Need[i])
4.     return "error";
5. if(Request>Available)
6.     sleep();
7. Available = Available - Request;
8. Allocation[i] = Allocation[i] + Request;
9. Need[i] = Need[i] - Request; /*先将资源分配给Pi*/
10. if(Banker()=="deadlock") /*调用银行家算法判定是否会死锁*/
11.     拒绝Request; /*若算法判定deadlock则拒绝请求，资源回滚*/
```

死锁避免之资源请求实例(1)

■ P1先申请资源(1,0,2)



	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
<i>P0</i>	0 1 0	7 4 3	2 3 0
<i>P1</i>	3 0 2	0 2 0	
<i>P2</i>	3 0 2	6 0 0	
<i>P3</i>	2 1 1	0 1 1	
<i>P4</i>	0 0 2	4 3 1	

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
<i>P0</i>	0 1 0	7 4 3	3 3 2
<i>P1</i>	2 0 0	1 2 2	
<i>P2</i>	3 0 2	6 0 0	
<i>P3</i>	2 1 1	0 1 1	
<i>P4</i>	0 0 2	4 3 1	

后续存在序列<P₁, P₃, P₂, P₄, P₀>是安全的
→所以此次申请允许

死锁避免之资源请求实例(2)

■ P0再申请资源(0,2,0)



	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
<i>P0</i>	0 3 0	7 2 3	2 1 0
<i>P1</i>	3 0 2	0 2 0	
<i>P2</i>	3 0 2	6 0 0	
<i>P3</i>	2 1 1	0 1 1	
<i>P4</i>	0 0 2	4 3 1	

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
<i>P0</i>	0 1 0	7 4 3	2 3 0
<i>P1</i>	3 0 2	0 2 0	
<i>P2</i>	3 0 2	6 0 0	
<i>P3</i>	2 1 1	0 1 1	
<i>P4</i>	0 0 2	4 3 1	

后续进程 P_0, P_1, P_2, P_3, P_4 一个也没法执行，死锁进程组
→所以此次P0申请被拒绝

- 每个进程进入系统时必须告知所需资源的最大数量对应用程序员要求高
- 安全序列寻找算法（安全状态判定算法）计算时间复杂度为 $O(mn^2)$ ，过于复杂
- 若每次资源请求都要调用银行家算法，耗时过大，系统效率降低
- 采用此算法，存在情况：当前有资源可用，尽管可能很快就会释放，由于会使整体进程处于不安全状态，而不被分配，致使资源利用率大大降低

死锁检测+恢复: 死锁检测

■ 基本原因: 每次申请都执行 $O(mn^2)$, 效率低

■ 对策: 只要可用资源足够满足**当前Request (不是Need)**, 则分配, 发现问题 (死锁) 再处理

➤ 定时检测或者当发现资源利用率低时检测 (注意这里是基于Request的检测)

```
1. bool Found;           //对银行家算法进行改进, 实现检测
2. int Request[n][m];
3. Work = Available; Finish = false;
4. if Allocation[i] != 0: Finish[i] = false;
5. else: Finish[i] = true;
6. while(true) {
7.     Found = false; //是否为安全序列找到一个新进程
8.     for(i=1; i<=n; i++){
9.         if(Finish[i]==false && Request[i]<=Work) {
10.             Work = Work + Allocation[i];
11.             Finish[i] = true;
12.             Found = true;
13.         }
14.     }
15.     if(Found==false)break;
16. }
```

对于无分配资源的进程, 不论其是否获得请求资源, 则认为其是完成的, 基于此来减少后续For循环遍历

```
17. for(i=1; i<=n; i++) {
18.     if(Finish[i]==false) {
19.         deadlock = deadlock + {i};
20.         return "deadlock";
21.     }
22. }
```

■ 终止进程 选谁终止?

➤ 优先级? 占用资源多的? ...

■ 剥夺资源 进程需要回滚 (rollback)

➤ 回滚点的选取? 如何回滚? ...

鸵鸟算法（死锁忽略）

■ 死锁预防？

- 引入太多不合理因素...

■ 死锁避免？

- 每次申请都执行银行家算法 $O(mn^2)$ ，效率太低

■ 死锁检测+恢复？

- 还要执行银行家算法 $O(mn^2)$ ，且恢复并不容易

■ 鸵鸟算法: 对死锁不做任何处理.....

- 死锁出现时，手动干预——重新启动
- 死锁出现不是确定的，避免死锁付出的代价毫无意义
- 有趣的是大多数操作系统都用它，如UNIX和Windows

- 进程竞争资源 $\Rightarrow$ 有可能形成循环竞争 $\Rightarrow$ 死锁
- 死锁需要处理 $\Rightarrow$ 死锁分析 $\Rightarrow$ 死锁的必要条件
- 死锁处理 $\Rightarrow$ 预防、避免、检测+恢复、忽略
- 死锁预防: 破除必要条件 $\Rightarrow$ 引入了不合理因素
- 死锁避免: 用银行家算法找安全序列 $\Rightarrow$ 效率太低
- 死锁检测恢复: 银行家算法找死锁进程组并恢复 $\Rightarrow$ 实现较难
- 死锁忽略: 就好像没有死锁 $\Rightarrow$ 现在用的最多

任何思想、概念、技术的主流都会随着时间而改变，操作系统尤为明显！

■ 1.死锁的避免是根据（）采取措施实现的。

A.配置足够的系统资源

B.使进程的推进顺序合理

C.破坏死锁的四个必要条件之一

D.防止系统进入不安全状态

■ 2.死锁预防是保证系统不进入死锁状态的静态策略，其解决方法是破坏产生死锁的四个必要条件之一。下列方法中破坏了“循环等待”条件的是：

A.银行家算法

B.一次性分配策略

C.剥夺资源法

D.资源有序分配

■ 3. 某系统中有三个并发进程都需要四个同类资源，则该系统必然不会发生死锁的最少资源数是（ ）

- A.9 B.10 C.11 D.12

- 4.某系统有R1、R2和R3共三种资源,在T0时刻P1、P2、P3和P4这四个进程对资源的占用和需求情况见下表,此时系统的可用资源向量为(2,1,2), 试问:
- 1) 用向量或矩阵表示系统中各种资源的总数和此刻各进程对各资源的需求数目。

进程	最大资源需求数			已分配资源数量		
	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0
P2	6	1	3	4	1	1
P3	3	1	4	2	1	1
P4	4	2	2	0	0	2

- 2) 若此时进程P1和进程P2均发出资源请求向量Request(1,0,1)，为了保证系统的安全性，应如何分配资源给这两个进程？说明所采用策略的原因。
- 3) 若2) 中两个请求立即得到满足后，系统此刻是否处于死锁状态？

Hope you enjoyed the OS course!